A. C. Patil College of Engineering
Kharghar, Navi Mumbai
Affiliated to University of Mumbai

WEB TECHNOLOGY LABORATORY MANUAL

Academic Year
2026–2027

Class	T.E. Information Technology
Semester	V
Subject	Web Technology Laboratory
Subject Code	2345117
Faculty Name	Sarin Shankar Deore
Academic Year	2026–2027

Department of Information Technology

A. C. Patil College of Engineering

Navi Mumbai, Maharashtra

2026–2027

Experiment No. 1

Aim

To design a responsive Registration Form using Bootstrap and perform client-side validation using JavaScript.

Introduction

Bootstrap is an open-source CSS framework used to develop responsive and mobile-first web applications. JavaScript is a client-side scripting language that enables validation, interactivity, and dynamic behavior in web pages. This experiment demonstrates form creation and input validation.

Code

registration.html

```
<!DOCTYPE html>
<html>
<head>
<title>Registration Form</title>

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">

<script>

function validate(){

let name=document.getElementById("name").value;
let email=document.getElementById("email").value;
let mobile=document.getElementById("mobile").value;

if(name==""){
alert("Enter Name");
return false;
}
```

```
if(email==""){
alert("Enter Email");
return false;
}

if(mobile.length!=10){
alert("Invalid Mobile Number");
return false;
}

alert("Registration Successful");
return true;

}

</script>

</head>

<body>

<div class="container mt-5">

<h2 class="text-center">Registration Form</h2>

<form onsubmit="return validate()">

<input type="text" id="name" class="form-control mb-3" placeholder="Enter Name">

<input type="email" id="email" class="form-control mb-3" placeholder="Enter Email">

<input type="text" id="mobile" class="form-control mb-3" placeholder="Enter Mobile Number">

<input type="submit" class="btn btn-primary">

</form>

</div>

</body>
</html>
```

Output

Registration Form

Name : _____

Email : _____

Mobile : _____

[Submit]

After successful validation

Registration Successful

Conclusion

The registration form was successfully designed using Bootstrap and validated using JavaScript.

Experiment No. 2

Aim

To demonstrate Arrow Functions and Anonymous Functions using JavaScript.

Introduction

JavaScript supports different types of functions. Arrow functions provide a concise syntax, whereas anonymous functions are functions without names and are mainly used as callback functions.

Code

```
<!DOCTYPE html>
<html>
<head>

<title>JavaScript Functions</title>

</head>

<body>

<h2>Arrow Function</h2>

<p id="demo1"></p>

<h2>Anonymous Function</h2>

<p id="demo2"></p>

<script>

const square=(num)=>num*num;

document.getElementById("demo1").innerHTML=
"Square of 5 = "+square(5);

let addition=function(a,b){

return a+b;

};

document.getElementById("demo2").innerHTML=
"Addition = "+addition(20,30);

</script>

</body>

</html>
```

Output

Arrow Function

Square of 5 = 25

Anonymous Function

Addition = 50

Conclusion

Arrow and anonymous functions were successfully implemented using JavaScript.

Experiment No. 3

Aim

To write JSX in React and develop a simple React application.

Introduction

JSX (JavaScript XML) is a syntax extension for JavaScript used in React. It allows developers to write HTML-like syntax inside JavaScript, making component creation easier and more readable.

Code

App.js

```
function App(){

return(

<div>

<h1>Welcome to React</h1>
```

```
<h2>Hello Students</h2>

</div>

);

}

export default App;
```

Output

Welcome to React

Hello Students

(or, in the browser)

Welcome to React

Hello Students

Conclusion

JSX was successfully used to create a simple React application.

Experiment No. 4

Aim

To implement Class Component and Functional Component in React.

Introduction

React applications are built using components. Components improve code reusability and maintainability. React provides two types of components: Class Components and Functional Components.

Code

```
import React from "react";

class Welcome extends React.Component{

render(){

return(

<h2>Class Component</h2>

);

}

}

function Student(){

return(

<h2>Functional Component</h2>

);

}

function App(){

return(

<div>

<Welcome/>

<Student/>

</div>
```

);

}

export default App;

Output

Conclusion

Both Class and Functional Components were successfully implemented in React, demonstrating component-based development.

Below is **Part 2 (Experiments 5–8)** of your **Web Lab Manual**. It includes only **Aim, Introduction, Code, Output, and Conclusion**, as requested. These experiments correspond to your uploaded experiment list.

Experiment No. 5

Aim

To implement **State** and **Props** in React.

Introduction

State is a built-in React object that stores component data and allows dynamic updates. Props are read-only properties passed from a parent component to a child component. They help create reusable and maintainable React components.

Code (App.js)

import React, { useState } from "react";

function Student(props){

```
    return <h2>Student Name : {props.name}</h2>;
}

function App(){

    const[count,setCount]=useState(0);

    return(
        <div>
            <Student name="Rahul"/>

            <h3>Counter : {count}</h3>

            <button onClick={()=>setCount(count+1)}>
                Increment
            </button>

        </div>
    );
}

export default App;
```

Output

Student Name : Rahul

Counter : 0

[Increment]

After clicking the button:

Student Name : Rahul

Counter : 1

[Increment]

Conclusion

State and Props were successfully implemented to create dynamic and reusable React components.

Experiment No. 6

Aim

To implement navigation using **React Router**.

Introduction

React Router is a standard routing library that enables navigation between multiple pages in a React application without refreshing the browser.

Code (App.js)

```
import {
BrowserRouter,
Routes,
Route,
Link
} from "react-router-dom";

function Home(){
return <h2>Home Page</h2>;
}

function About(){
return <h2>About Page</h2>;
}

function Contact(){
return <h2>Contact Page</h2>;
}

function App(){

return(
```

```
<BrowserRouter>

<nav>

<Link to="/">Home</Link> |

<Link to="/about">About</Link> |

<Link to="/contact">Contact</Link>

</nav>

<Routes>

<Route path="/" element={<Home/>}/>

<Route path="/about" element={<About/>}/>

<Route path="/contact" element={<Contact/>}/>

</Routes>

</BrowserRouter>

);

}

export default App;
```

Output

Home | About | Contact

Home Page

When **About** is clicked

Home | About | Contact

About Page

When **Contact** is clicked

Home | About | Contact

Contact Page

Conclusion

Navigation between multiple pages was successfully implemented using React Router.

Experiment No. 7

Aim

To install MongoDB and create a database, collections, and documents.

Introduction

MongoDB is a NoSQL document-oriented database. It stores data in BSON (Binary JSON) format and provides flexible schema design for modern web applications.

Code

```
show dbs

use college

db.createCollection("student")

db.student.insertOne({

roll:101,

name:"Rahul",

branch:"IT"

})
```

db.student.find()

Output

test

admin

college

switched to db college

{ "ok" : 1 }

{
 "_id":ObjectId("..."),
 "roll":101,
 "name":"Rahul",
 "branch":"IT"
}

Conclusion

MongoDB was successfully installed, and a database, collection, and document were created.

Experiment No. 8

Aim

To perform CRUD (Create, Read, Update, Delete) operations using MongoDB.

Introduction

CRUD operations are the fundamental database operations used in MongoDB. They allow users to create, retrieve, modify, and delete documents efficiently.

Code

```
use college

db.student.insertOne({

roll:102,

name:"Amit",

branch:"IT"

})

db.student.find()

db.student.updateOne(

{roll:102},

{$set:{branch:"Computer"}}

)

db.student.deleteOne(

{roll:102}

)

db.student.find()
```

Output

Document Inserted Successfully

```
{
roll:102,
name:"Amit",
branch:"IT"
}
```

Document Updated Successfully

{
roll:102,
name:"Amit",
branch:"Computer"
}

Document Deleted Successfully

No Documents Found

Conclusion

CRUD operations were successfully performed in MongoDB using insert, find, update, and delete commands.

Below is Part 3 (Experiments 9–12) of your Web Lab Manual. It includes only Aim, Introduction, Code, Output, and Conclusion as requested. These experiments correspond to your uploaded Web Lab experiment list.

Experiment No. 9

Aim

Part-A: To perform arithmetic operations using Node.js REPL.

Part-B: To create and run a basic HTTP server using Node.js.

Introduction

Node.js is a JavaScript runtime built on Chrome's V8 engine. It enables developers to build scalable server-side applications. The REPL (Read-Evaluate-Print Loop) provides an interactive environment to execute JavaScript statements.

Code

Part-A (Node REPL)

```
> 20+30

> 50-15

> 12*5

> 100/4
```

Part-B (server.js)

```
const http=require("http");

const server=http.createServer((req,res)=>{

res.writeHead(200,{'Content-Type':'text/html'});

res.write("<h1>Welcome to Node.js Server</h1>");

res.end();

});

server.listen(3000);

console.log("Server Running at Port 3000");
```

Output

Node REPL

```
>20+30
50

>50-15
35

>12*5
60

>100/4
25
```

Browser

Welcome to Node.js Server

Terminal

Server Running at Port 3000

Conclusion

Arithmetic operations were executed successfully using Node REPL, and a basic HTTP server was created using Node.js.

Experiment No. 10

Aim

To demonstrate File System, Buffers, and Streams in Node.js.

Introduction

The File System module provides APIs to create, read, update, and delete files. Buffers handle binary data, while Streams efficiently process large amounts of data without loading the entire file into memory.

Code

```
const fs=require("fs");

fs.writeFileSync("demo.txt","Welcome to Node.js");
```

```
let data=fs.readFileSync("demo.txt","utf8");

console.log(data);

let buffer=Buffer.from("Hello");

console.log(buffer);

const readStream=fs.createReadStream("demo.txt");

readStream.on("data",(chunk)=>{

console.log(chunk.toString());

});
```

Output

Welcome to Node.js

<Buffer 48 65 6c 6c 6f>

Welcome to Node.js

Conclusion

The File System module, Buffers, and Streams were successfully demonstrated using Node.js.

Experiment No. 11

Aim

To build a REST API using Express.js and test the API using Postman.

Introduction

Express.js is a lightweight web framework for Node.js that simplifies the development of RESTful web services. Postman is an API testing tool used to send HTTP requests and verify responses.

Code

```
const express=require("express");

const app=express();

app.get("/",(req,res)=>{

res.send("Welcome to Express API");

});

app.get("/student",(req,res)=>{

res.json({

roll:101,

name:"Rahul",

branch:"IT"

});

});

app.listen(3000,()=>{

console.log("Server Running");

});
```

Output

Browser

http://localhost:3000

Welcome to Express API

Browser/Postman

http://localhost:3000/student

```
{
  "roll":101,
  "name":"Rahul",
  "branch":"IT"
}
```

Terminal

Server Running

Conclusion

A REST API was successfully developed using Express.js and tested using Postman.

Experiment No. 12

Aim

To connect a React frontend with a Node.js backend using Fetch API and display dynamic data.

Introduction

Modern web applications consist of frontend and backend components. React communicates with backend services using HTTP requests through Fetch API or Axios to display dynamic information.

Backend Code (server.js)

```
const express=require("express");
```

```
const cors=require("cors");

const app=express();

app.use(cors());

app.get("/student",(req,res)=>{

res.json({

name:"Rahul",

branch:"IT"

});

});

app.listen(5000);
```

Frontend Code (App.js)

```
import React,{useEffect,useState} from "react";

function App(){

const[data,setData]=useState({});

useEffect(()=>{

fetch("http://localhost:5000/student")

.then(res=>res.json())

.then(result=>setData(result));

},[]);

return(

<div>
```

<h2>{data.name}</h2>

<h3>{data.branch}</h3>

** **

);

}

export default App;

Output

Browser

Rahul

IT

Backend Terminal

Server Running at Port 5000

Conclusion

The React frontend was successfully connected with the Node.js backend using the Fetch API, and dynamic data was retrieved and displayed.
